Hybrid RAG Support Assistant: App Creation & Deployment Manual

A Step-by-Step Blueprint: Project Artifacts, Code Provenance, Architecture, and Cloud Deployment

Executive Summary: This document provides the complete technical explanation of how the **Hybrid RAG Customer Support Assistant** was extracted from experimental Jupyter Notebooks and turned into a standalone, production-ready Streamlit web application. It specifies every file required, traces where each line of code originated in your development pipeline, and provides exact steps to deploy the live app on **Streamlit Community Cloud** and **Hugging Face Spaces** for recruiters.

1. System Architecture & Dual-Role LLM Design

Standard RAG passes user queries directly to a vector database, frequently retrieving noisy or irrelevant context. This project introduces a **Hybrid RAG pipeline** that uses a single backbone model (**Qwen/Qwen2.5-0.5B-Instruct**) operating in two coordinated phases:

Phase 1: Intent Routing (LoRA Active)	The user query is passed to the LLM with the LoRA adapter enabled. The adapter classifies the intent and category into a strict JSON payload (e.g., {"intent": "cancel_order", "category": "order"}).
Phase 2: Semantic Retrieval (ChromaDB)	The extracted intent maps to a target Standard Operating Procedure (SOP). sentence-transformers/all-MiniLM-L6-v2 queries ChromaDB to retrieve top-k grounded chunks with similarity scoring.
Phase 3: Grounded Answer (LoRA Disabled)	The LoRA adapter is toggled off in memory. The base model synthesizes a customer support answer strictly conditioned on the retrieved SOP context, preventing hallucinations.

2. Required Files Inventory for the Deployment App

To deploy the app to Hugging Face or Streamlit Cloud, you only need the self-contained **hybrid_rag_app/** folder. Large raw training corpora (1+ GB) and virtual environments (.venv/) are excluded, reducing the bundle size to only **~26 MB**.

File / Folder	Size	Role & Description
app.py	~6 KB	Main Streamlit application. Manages session state, chat bubbles, intent badge display, retrieved context expanders, and streaming tokens.
rag/pipeline.py	~5 KB	Pipeline orchestrator: coordinates user input → intent classification → query formulation → SOP retrieval → grounded response.
rag/llm.py	~4 KB	Dual-role model manager: dynamically attaches/detaches LoRA adapter on Qwen2.5-0.5B. Uses bfloat16 on CPU to halve RAM.
rag/retriever.py	~3 KB	ChromaDB vector retriever: connects to persistent collection and embeds queries with all-MiniLM-L6-v2.
rag/config.py	~2 KB	Central configuration: default hyperparameters, model IDs, file paths, and the 27 INTENT_TO_DOC mapping rules.
models/intent_lora_best/	~8.6 MB	Trained LoRA adapter weights (adapter_model.safetensors + adapter_config.json) fine-tuned for intent routing.

chroma_db/	~1.1 MB	Persistent ChromaDB vector index containing vectorized SOP policy chunks across 10 business domains.
requirements.txt	~260 B	Dependencies file specifying CPU-optimized PyTorch, Transformers, PEFT, ChromaDB, and Streamlit.
Dockerfile	~350 B	Container specification for Hugging Face Spaces (exposes port 8501, runs Streamlit in Python 3.10-slim).
README.md	~3.8 KB	Space configuration YAML header for Hugging Face + full recruiter-ready project documentation.

3. Code Provenance: Where Each Component Came From

Every file in the standalone app directly derives from your experimental Jupyter notebooks and data preparation pipeline. Below is the exact lineage mapping:

App Component	Source Notebook / Script	Original Implementation Details
LoRA Adapter (models/intent_lora_best/)	Fine_Tuning_Pipeline.ipynb (and scripts/train_lora.py)	Trained on Bitext Customer Support dataset with HuggingFace PEFT. Configured with r=16, lora_alpha=32, targeting modules [q_proj, k_proj, v_proj, o_proj] on base model Qwen2.5-0.5B-Instruct.
Vector Index (chroma_db/)	RAG_Implementation.ipynb (and scripts/rebuild_index.py)	Extracted SOP texts from Dataset/ into retrieval_corpus.json. Chunked into 250-token windows and embedded using all-MiniLM-L6-v2 into collection sop_knowledge_base.
Intent Routing Logic (rag/llm.py)	Solution_V2_FineTuned_RAG_Evaluation.ipynb	Implements dynamic adapter toggling (model.enable_adapters() and model.disable_adapters()) so a single model instance handles both structured classification and open generation.
Intent-to-Doc Map (rag/config.py)	Data_Preparation.ipynb & SOP Schema	Maps the 27 Bitext intent labels (e.g. track_order, get_refund, recover_password) to the 10 domain SOP markdown files (e.g. order_tracking, refund_policy).
Retriever Logic (rag/retriever.py)	RAG_Implementation.ipynb	Wraps ChromaDB's query() function with cosine similarity thresholding and chunk deduplication.
End-to-End Flow (rag/pipeline.py)	Solution_V2_FineTuned_RAG_Evaluation.ipynb	Implements query formulation: constructs targeted retrieval strings based on the extracted intent before searching ChromaDB, providing superior precision over raw query retrieval.
Web UI & Badges (app.py)	Production Streamlit Layer	Provides UI controls, real-time message streaming via TextIteratorStreamer, intent/category badges, and expandable source attribution panels.

4. Step-by-Step Deployment Guide

You have two reliable, free options to host your app online so recruiters can test it via a live URL:

Option A: Deploy to Streamlit Community Cloud (Fastest & Guaranteed)

1. **Push your clean app to GitHub:** Run the following in your PowerShell terminal:

```
cd "c:\Users\HP\OneDrive\Desktop\Rag_Old_New\complete_final _project\Rag_final_files\hybrid_rag_app" git push origin main
```

2. **Connect on Streamlit Cloud:**
   - Open **share.streamlit.io** and sign in with GitHub.
   - Click **New app** and select Rakeshpawade/hybrid-rag-intent-lora.
   - Main file path: app.py | Branch: main.
   - Click **Deploy!** — Streamlit Cloud will launch your app with a clean *.streamlit.app URL.

Option B: Deploy to Hugging Face Spaces (16 GB Free RAM)

1. **Create a Space on Hugging Face:**
   - Go to **huggingface.co/new-space**.
   - Space SDK: choose **Docker** (Blank template). Hardware: **CPU basic · 16 GB · Free**.

2. Push or upload files:

- Either drag-and-drop the hybrid_rag_app files under the **Files** tab, or push via git:

```
git remote add space https://huggingface.co/spaces/YOUR_USERNAME/hybrid-rag-support.git git push space main -f
```

5. Recruiter Presentation & Resume Section

Copy and paste this structured bulleted entry into the **Projects** section of your CV / Resume:

Hybrid RAG & LoRA Customer Support Assistant | Python, PyTorch, LoRA, ChromaDB, Streamlit
Live App: [Your-Streamlit-Or-HF-Link] • *GitHub: github.com/Rakeshpawade/hybrid-rag-intent-lora*
• Architected a dual-role customer support assistant using an open-source Qwen-0.5B causal language model.
• Fine-tuned a parameter-efficient LoRA adapter for structured intent classification and domain entity extraction.
• Developed a dense semantic retrieval pipeline with ChromaDB and MiniLM embeddings over domain SOP manuals.
• Engineered an interactive Streamlit UI with real-time token streaming, intent badges, and source attribution.